Point 05 — Software Design

Project: Electrical Consumption Monitor — EnergyMonitor **Record:** 3145556 — Software Analysis and Development (ADSO), SENA CIES Regional Huila **Members:** Juan Esteban Cortés Parra · Miguel Ángel García Artunduaga **Document version:** 1.0

> **Update note (10/27/2026).** This document describes the design of version 1.0, when the backend was the `energy-monitor-backend` modular monolith (Spring Modulith). That design evolved: the seven bounded contexts are now **seven microservices** behind the `en-monitor-api-gateway` API gateway (8080) — security 8082, device 8084, measurement 8085, home 8086, notification 8087, alert 8088 and recommendation 8089 —, communicating through internal REST (`/internal/**`) and RabbitMQ events with a transactional outbox, on the same shared MySQL schema (37 tables). The modular backend was retired. The hexagonal layers, contexts and patterns described here live on inside each service; the current architecture, routes, events and configuration are in the technical manual (sections 9 to 13) and the current data model in `06-base-datos`.

1. Purpose of point 05

Software analysis (point 03) determines **what** the system must do and **how it should look**. Software design answers **how it is built**: what structure the system has internally, how its data is organized, which patterns are used to solve each recurring problem, and how the visual design is materialized into reusable components.

This point is the bridge between analysis and implementation. Every design decision documented here is backed by existing code in the project's three repositories, not by merely theoretical decompositions.

2. Documents in this point

Document	Content	Question it answers
`05.1-architectural-model.md`	Architectural style, layers, bounded contexts, components, deployment, quality attributes and verifiable rules	**How is the system organized?**
`05.2-data-model.md`	29 tables, conventions, cross-context references, integrity, Liquibase and differences with the ER model of point 03	**How is the data stored?**
`05.3-class-model.md`	Classes, interfaces, enumerations, relationships and differences with the class diagram of point 03	**How does the system behave internally?**
`05.4-design-patterns.md`	18 documented patterns with problem, application, benefit and cost	**Which solutions were applied and why?**
`05.5-graphic-interfaces.md`	Design system, palette, typography, screens, components, interaction, accessibility and i18n	**How does it look and how is it used?**

3. Executive summary

Aspect	Definition
Architectural style	Layered client-server + API gateway + 7 microservices with hexagonal architecture (version 1.0: Spring Modulith modular monolith, now retired)
Presentation context	React 19 + Vite 8 (web) and Expo SDK 57 (mobile), with its own design system on both
Logic	Java 21 + Spring Boot 4.1.1, organized into **7 bounded contexts** with boundaries verified by tests
Data	MySQL 8.4, shared schema with **37 tables** (29 business tables in version 1.0, plus outboxes and pending registrations), each service owning its own, Liquibase and soft delete

Aspect	Definition
Communication	Stateless REST API under `/api/v1` with JWT RS256; device telemetry over MQTT
Patterns	18 documented patterns, with emphasis on ports and adapters, Observer, Strategy and Repository
Interface	Own design system with 32 tokens, 4 themes, 11 typographic sizes, 4 languages
Tests	~120 test files in the backend (6 levels) · 13 suites on the web · 0 on mobile

4. Repository structure

Repository	Technology	Entry point
`en-monitor-api-gateway` and seven `en-monitor-*-service`	Java 21 + Spring Boot 4.1.1 (gateway 4.0.8) + Maven	`*Application.java` (gateway 8080; services 8082 and 8084 to 8089)
`energy-monitor-backend` (retired)	Java 21 + Spring Modulith	No longer runs; keeps the firmware
`Energy-Monitor-application`	React 19 + Vite 8 (JavaScript)	`src/main.jsx` over `index.html` (port 5173)
`Energy-Monitor-Mobile`	Expo SDK 57 + React Native 0.86 (TypeScript)	`src/app/_layout.tsx`

5. Graphic resources

File	Content
`assets/diagramas/MER.wsd`	Entity-relationship model in PlantUML syntax (artifact of point 03)
`assets/diagramas/Diagrama_clases.wsd`	Class diagram in PlantUML syntax (artifact of point 03)

The files are rendered at https://www.plantuml.com/plantuml/uml or with the PlantUML plugin for Visual Studio Code.

6. Relationship with the other points of the project

6.1 Point 03 — Software analysis (origin of this point)

Point 03 artifact	What it contributes to point 05
Mockups (web and mobile)	Define the visual look of each screen → `05.5-graphic-interfaces.md`
Navigation maps	Define the screen tree → `05.5-graphic-interfaces.md`, sections 4 and 5
User stories and use cases (point 02)	Define the functionalities → `05.1-architectural-model.md`, section 3
ER model	Defines the data model → `05.2-data-model.md`
Class diagram	Defines the class model → `05.3-class-model.md`

> **The mockups live in point 03.** This point describes them and relates them to the code, but does not duplicate them.

6.2 Point 02 — Requirements specification

Requirement	Where it is materialized
RF-SEC User and Profile Management	`security` context · access and account screens
RF-HOM Household Management	`home` context · home panel and home detail
RF-HOM-08, RF-NOT-03, RF-CLI-05, RF-CLI-06 Thresholds, preferences, language and colors	`home_thresholds` + `user_configuration` · configuration screen

Requirement	Where it is materialized
RF-MED, RF-ALE, RF-REC, RF-NOT Measurements, alerts, recommendations and notifications	`measurement`, `alert`, `recommendation` contexts · consumption and history tabs
RF-DEV Device Management (connection status: RF-MED-04 to RF-MED-06)	`device` context · devices tab and linking wizard
RNF-SEG, RNF-REN, RNF-DIS, RNF-COM, RNF-MAN, RNF-I18, RNF-ESC, RNF-USA, RNF-INT	`05.1-architectural-model.md`, section 7

6.3 Point 04 — Technical proposal

The technical proposal defines **with what** the system is built. This point defines **how** it is structured. The technical decisions made in point 04 (languages, frameworks, tools) are applied here in a concrete way.

6.4 Later points

Point	What it takes from this point
06 — Database	Data dictionary and database scripts derived from the data model
07 — Tests	Test cases derived from the integrity rules and the module boundaries
08 — Quality and improvements	Gaps recorded in sections 9 and 10 of each document
09 — Implementation	Traceability between the design documents and the build tasks
10 — Technical manual	Architecture, folder structure and configuration
12 — User manual	Screens, components and messages defined in `05.5-graphic-interfaces.md`

7. Version-control observations

7.1 Changes between the design artifact (point 03) and the implemented code

During the construction of the software, the schema and the class model evolved. This point documents **the current state** and records the differences in sections 9 (`05.2-data-model.md`) and 10 (`05.3-class-model.md`).

Document	Section	Content
`05.2-data-model.md`	9	13 differences between the point 03 ER model and the current schema
`05.3-class-model.md`	10	8 differences between the point 03 class diagram and the current code

Most relevant differences:

#	Difference	Impact
1	The published ER model does not include every table; the current schema has 29	`notification_preference` and `push_subscription` were added
2	The note in the class diagram states that the Strategy pattern is not implemented	**All three classes do exist** in `home/domain/model`
3	The class diagram models `RecommendationAnalyzer`	The code uses `RecommendationRules` (domain service)
4	`AlertType` had two values	The current schema has four

Recommended action: update `MER.wsd` and `Diagrama_clases.wsd` in `05-diseno-software/EN/assets/diagramas/` before the final delivery.

7.2 Recorded design gaps

Document	Section	Number of gaps
`05.2-data-model.md`	10	5 pending decisions

Document	Section	Number of gaps
`05.4-design-patterns.md`	8	5 decisions not to apply a pattern
`05.5-graphic-interfaces.md`	7.2	6 accessibility gaps
`05.5-graphic-interfaces.md`	10	10 pending interface tasks

These gaps are documented on purpose: the program guidelines recommend describing the software **as it is built**, without hiding what is missing. Each one carries its own solution recommendation.

8. How to use this point

If you are…	Start with…
Sheet evaluator	Section 3 of this index and section 7 of `05.1-architectural-model.md`
New developer on the project	`5.1` → `5.4` → the code of the module assigned to you
Database designer	`05.2-data-model.md` and `05.3-class-model.md`
Interface designer	`05.5-graphic-interfaces.md` and the mockups from point 03
Project manager	The gap sections: 5.2 §10, 5.4 §8, 5.5 §7.2 and §10